

Edge cases (a missing required argument) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# missing_argument_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a named city. Requires a city name."""
    return f"Sunny in {city}"

model_with_tools = ChatOpenAI().bind_tools([get_weather])

for run_number in range(1, 6):
    response = model_with_tools.invoke("what's the weather like")

    if not response.tool_calls:
        print(f"Run {run_number}: asked or answered directly -> {response.content}")
        continue

    city_sent = response.tool_calls[0]["args"].get("city")
    if not city_sent:
        print(f"Run {run_number}: called with an empty city value")
    else:
        print(f"Run {run_number}: called with city = {city_sent!r}")
```

Expected output (an example split — real model behavior varies run to run):

```
Run 1: asked or answered directly -> Sure – which city would you like the weather
Run 2: called with city = 'London'
Run 3: asked or answered directly -> Could you tell me which city you mean?
```

```
Run 4: called with city = 'New York'  
Run 5: called with an empty city value
```

This shows you the real split across 5 runs. It doesn't save the results anywhere, so once the run finishes you've only got what scrolled past in the terminal — fine for a quick look, not for writing up what you found.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — classified and returned as structured records

```
# missing_argument_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a named city. Requires a city name."""
    return f"Sunny in {city}"

def get_model_with_tools() -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([get_weather])

def classify_response(response) -> str:
    if not response.tool_calls:
        return "asked_or_answered_directly"
    city_sent = response.tool_calls[0]["args"].get("city")
    if not city_sent:
        return "called_with_empty_value"
    return "called_with_guessed_value"

def run_missing_argument_trial(prompt: str, runs: int = 5) -> list[dict]:
    model_with_tools = get_model_with_tools()
    records = []

    for run_number in range(1, runs + 1):
        response = model_with_tools.invoke(prompt)
        outcome = classify_response(response)
        record = {"run": run_number, "outcome": outcome}
        if outcome == "called_with_guessed_value":
            record["city_sent"] = response.tool_calls[0]["args"].get("city")
        if outcome == "asked_or_answered_directly":
            record["reply"] = response.content
        records.append(record)
```

```
    return records

if __name__ == "__main__":
    results = run_missing_argument_trial("what's the weather like")
    for record in results:
        print(record)
```

Expected output (example):

```
{'run': 1, 'outcome': 'asked_or_answered_directly', 'reply': 'Which city would you like?'}
{'run': 2, 'outcome': 'called_with_guessed_value', 'city_sent': 'London'}
{'run': 3, 'outcome': 'asked_or_answered_directly', 'reply': 'Could you specify a city?'}
{'run': 4, 'outcome': 'called_with_guessed_value', 'city_sent': 'New York'}
{'run': 5, 'outcome': 'called_with_empty_value'}
```

Difference from Basic: returning a list of typed records instead of only printing lets you actually count the split afterward (how many asked vs. guessed vs. something else) rather than having to re-read scrolled-past terminal output. Separating `classify_response()` out also makes it independently testable against a fake `response` object, no real API call needed. Neither version yet stops an empty-string guess from being accepted as a "real" value — that's what Advanced adds.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a Pydantic `args_schema` that rejects an empty guess

```
# missing_argument_practice.py
from pydantic import BaseModel, Field, ValidationError

from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

class WeatherArgs(BaseModel):
    city: str = Field(min_length=1, description="The city to get weather for.")

@tool(args_schema=WeatherArgs)
def get_weather(city: str) -> str:
    """Get the current weather for a named city. Requires a city name."""
    return f"Sunny in {city}"

def get_model_with_tools() -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([get_weather])

def classify_response(response) -> dict:
    if not response.tool_calls:
        return {"outcome": "asked_or_answered_directly", "reply": response.content}

    args_sent = response.tool_calls[0]["args"]
    try:
        WeatherArgs(**args_sent)
    except ValidationError as e:
        return {"outcome": "rejected_by_validation", "error": str(e)}

    return {"outcome": "called_with_value", "city_sent": args_sent.get("city")}

def run_missing_argument_trial(prompt: str, runs: int = 5) -> list[dict]:
    model_with_tools = get_model_with_tools()
    records = []
```

```

for run_number in range(1, runs + 1):
    response = model_with_tools.invoke(prompt)
    record = {"run": run_number, **classify_response(response)}
    records.append(record)

return records

if __name__ == "__main__":
    results = run_missing_argument_trial("what's the weather like")
    for record in results:
        print(record)

```

Expected output (example — note run 5's empty guess is now a caught validation failure, not a silent "empty value" label):

```

{'run': 1, 'outcome': 'asked_or_answered_directly', 'reply': 'Which city would you like?'}
{'run': 2, 'outcome': 'called_with_value', 'city_sent': 'London'}
{'run': 3, 'outcome': 'asked_or_answered_directly', 'reply': 'Could you specify a city?'}
{'run': 4, 'outcome': 'called_with_value', 'city_sent': 'New York'}
{'run': 5, 'outcome': 'rejected_by_validation', 'error': "1 validation error for WeatherArgs"}

```

Approach 2 — flagging suspicious "confident wrong guess" values

`min_length=1` catches an empty string, but it can't catch a *plausible-looking* guess like `"London"` when the user never said London. This approach doesn't try to fully solve that (nothing can, reliably) — it flags a small set of common demo/default values as suspicious, so at least the *known* guessing patterns get surfaced instead of silently passing as real input:

```

# missing_argument_practice.py
SUSPICIOUS_DEFAULT_CITIES = {"london", "new york", "n/a", "unknown", "test"}

def classify_response(response) -> dict:
    if not response.tool_calls:
        return {"outcome": "asked_or_answered_directly", "reply": response.content}

    args_sent = response.tool_calls[0]["args"]
    try:
        WeatherArgs(**args_sent)
    except ValidationError as e:
        return {"outcome": "rejected_by_validation", "error": str(e)}

    city_sent = args_sent.get("city", "")

```

```

if city_sent.strip().lower() in SUSPICIOUS_DEFAULT_CITIES:
    return {"outcome": "suspicious_default_guess", "city_sent": city_sent}

return {"outcome": "called_with_value", "city_sent": city_sent}

```

Expected output on a run that guesses a common default:

```
{'run': 2, 'outcome': 'suspicious_default_guess', 'city_sent': 'London'}
```

This is a heuristic, not a guarantee — a genuinely London-based user asking about "the weather" would trigger the same flag. It's worth having anyway in a tool that logs its own calls, since "the model called this with London 40% of the time on prompts that never mentioned a city" is exactly the kind of signal that tells you a prompt or tool description needs a rethink.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate classifies what happened, but treats an empty-string guess the same as any other "the model called the tool" outcome — nothing actually stops it. Approach 1 makes the empty case a real, caught validation failure at the tool's own contract boundary, so no caller anywhere has to remember to check for it by hand. Approach 2 tackles the harder, unsolved half of the problem — a confident-*looking* wrong guess — with an honest heuristic rather than a false promise of catching it reliably.

Which one should you actually write? Approach 1's `Field(min_length=1)` constraint costs almost nothing and belongs on essentially every required string argument you ever give a tool — there's no real downside to rejecting an empty guess outright. Approach 2's suspicious-default heuristic is worth adding only once you actually have telemetry showing a specific tool guesses specific values often — build the list from what you observe, not from guessing in advance what a model might guess.